

Practical no:1

Aim:

Breadth First Search & Iterative Depth First Search

Implement the Breadth First Search algorithm to solve a given problem.

Implement the Iterative Depth First Search algorithm to solve the same problem.

Compare the performance and efficiency of both algorithms.

```
from collections import deque
import time

# 1. Representing the map as an adjacency list (Graph) Carol Pillai S104
andheri_map = {
    'Seven Hills Hospital': ['Andheri-Kurla Road', 'Marol Naka', 'Chakala Junction'],

    'Marol Naka': ['Marol Church Road'],
    'Marol Church Road': ['Andheri Airport Road'],
    'Andheri Airport Road': ['Sahar Road Junction'],

    'Chakala Junction': ['J B Nagar Road'],
    'J B Nagar Road': ['Sahar Road Junction'],

    'Andheri-Kurla Road': ['Sakinaka Junction'],
    'Sakinaka Junction': ['Sahar Road Junction'],

    'Sahar Road Junction': ['Andheri Station East'],
    'Andheri Station East': ['A S Marg'],
    'A S Marg': ['Azad Nagar'],

    'Azad Nagar': ['MVLU College'],
    'MVLU College': []
}

# A) Breadth First Search (BFS)
def bfs_shortest_path(graph, start, destination):
    queue = deque([(start, [start])])
    visited = {start}
    nodes_explored = 0
```

```
while queue:
    current, path = queue.popleft()
    nodes_explored += 1

    if current == destination:
        return path, nodes_explored

    for neighbor in graph.get(current, []):
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append((neighbor, path + [neighbor]))

    return None, nodes_explored

# B) Iterative Depth First Search (DFS)
def iterative_dfs_path(graph, start, destination):
    stack = [(start, [start])]
    visited = set()
    nodes_explored = 0

    while stack:
        current, path = stack.pop()
        nodes_explored += 1

        if current == destination:
            return path, nodes_explored

        if current not in visited:
            visited.add(current)
            for neighbor in reversed(graph.get(current, [])):
                if neighbor not in visited:
                    stack.append((neighbor, path + [neighbor]))

    return None, nodes_explored

# C) Compare performance
if __name__ == "__main__":
```

```

start_node = 'Seven Hills Hospital'
end_node = 'MVLU College'

t0 = time.perf_counter()
bfs_path, bfs_nodes = bfs_shortest_path(andheri_map, start_node,
end_node)
t1 = time.perf_counter()
bfs_time = t1 - t0

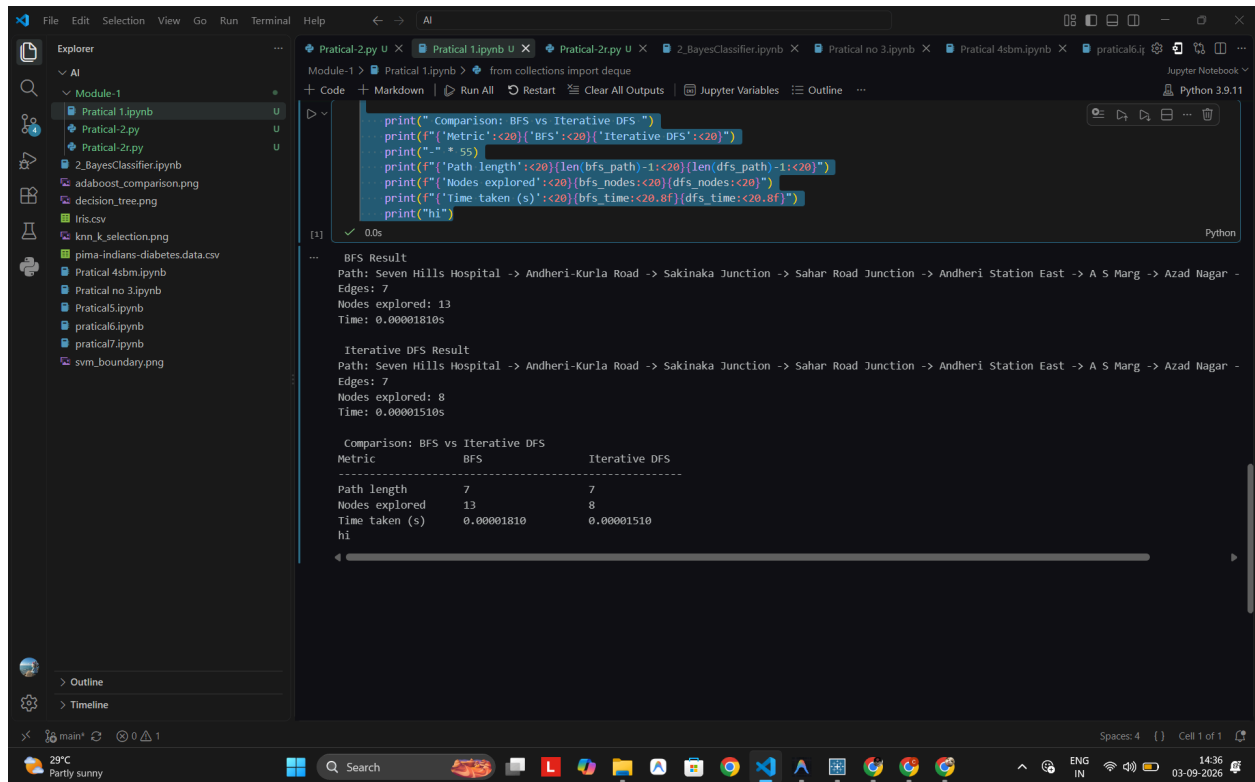
t2 = time.perf_counter()
dfs_path, dfs_nodes = iterative_dfs_path(andheri_map, start_node,
end_node)
t3 = time.perf_counter()
dfs_time = t3 - t2

print(" BFS Result ")
print("Path:", " -> ".join(bfs_path))
print("Edges:", len(bfs_path) - 1)
print("Nodes explored:", bfs_nodes)
print(f"Time: {bfs_time:.8f}s\n")

print(" Iterative DFS Result ")
print("Path:", " -> ".join(dfs_path))
print("Edges:", len(dfs_path) - 1)
print("Nodes explored:", dfs_nodes)
print(f"Time: {dfs_time:.8f}s\n")

print(" Comparison: BFS vs Iterative DFS ")
print(f"{'Metric':<20}{'BFS':<20}{'Iterative DFS':<20}")
print("-" * 55)
print(f"{'Path'
length':<20}{len(bfs_path)-1:<20}{len(dfs_path)-1:<20}")
print(f"{'Nodes explored':<20}{bfs_nodes:<20}{dfs_nodes:<20}")
print(f"{'Time taken (s)':<20}{bfs_time:<20.8f}{dfs_time:<20.8f}")
print("hi")

```



```
print(" Comparison: BFS vs Iterative DFS ")
print(f"{'Metric':<20}{'BFS':<20}{'Iterative DFS':<20}")
print("-" * 55)
print(f"{'Path length':<20}{len(bfs_path)-1:<20}{len(dfs_path)-1:<20}")
print(f"{'Nodes explored':<20}{bfs_nodes:<20}{dfs_nodes:<20}")
print(f"{'Time taken (s)':<20}{bfs_time:<20.8f}{dfs_time:<20.8f}")
print("hi")
```

[1]: 0.0s

BFS Result
Path: Seven Hills Hospital -> Andheri-Kurla Road -> Sakinaka Junction -> Sahar Road Junction -> Andheri Station East -> A S Marg -> Azad Nagar -
Edges: 7
Nodes explored: 13
Time: 0.00001810s

Iterative DFS Result
Path: Seven Hills Hospital -> Andheri-Kurla Road -> Sakinaka Junction -> Sahar Road Junction -> Andheri Station East -> A S Marg -> Azad Nagar -
Edges: 7
Nodes explored: 8
Time: 0.00001510s

Comparison: BFS vs Iterative DFS		
Metric	BFS	Iterative DFS
Path length	7	7
Nodes explored	13	8
Time taken (s)	0.00001810	0.00001510
hi		